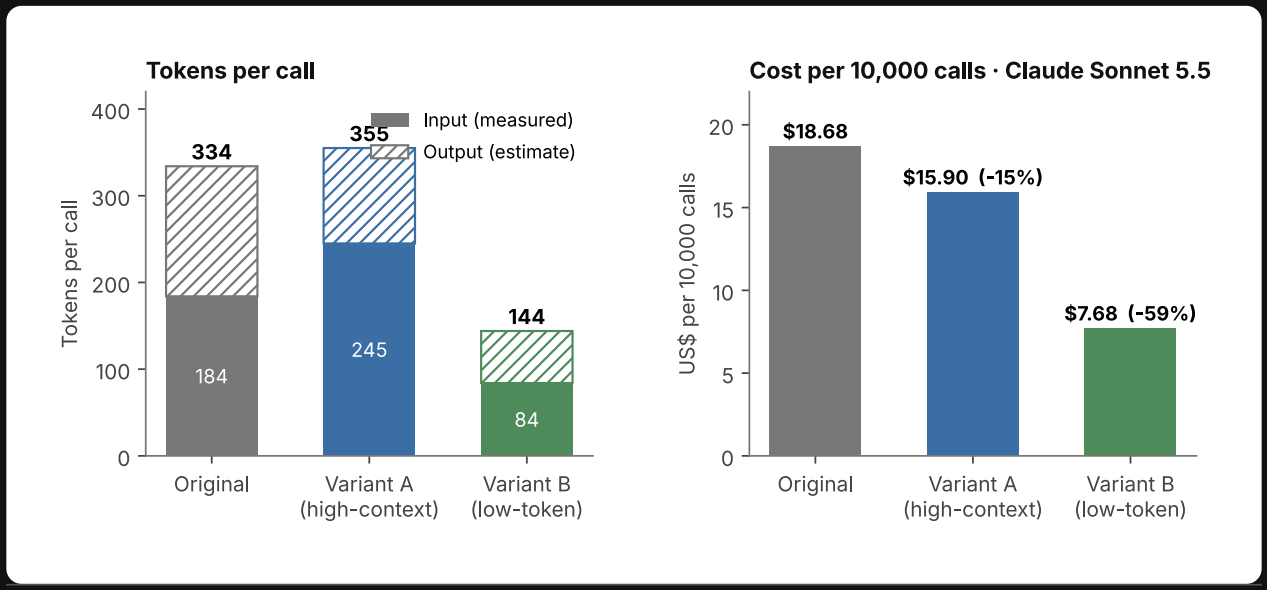
PRODUCT REQUIREMENT DOCUMENT · V1.1

OptimusPrompt

Rewrites a long, chatty prompt into two shorter, checked versions — one built for accuracy, one built for cost — and proves nothing important was lost.



Team	Date	Research base	Reference model
Wrost Case	24 October 2026	13 published papers	Claude Sonnet 5.5

Contents

1 Executive summary 4

2 The problem and the people who have it 6

3 How people will use it 9

4 What we will build, and in what order 12

5 How it works 14

6 What makes it different 23

7 How we will measure success 26

8 Assumptions, limits and risks 28

A The worked example in full 30

B Starting settings 33

C References 34

> **HOW TO READ THE NUMBERS**
>
> Every figure in this report is one of three things: measured on our worked example, taken from a cited paper (shown as [P1], [P4] and so on), or a starting setting we chose and will tune — those are listed in Appendix B. Answer-length figures for the worked example are estimates until the benchmark in Section 7 is run, and we say so wherever they appear.

List of figures

1	Tokens and cost for the worked example (cover)	13	The four checks, in order
2	Where the money goes	14	Recommendation rules
3	Removing 20–35% of context is almost free; past 50%, refusals climb [P4]	15	The cascade
4	The worked example after step 2	16	Request and response at a glance
5	Build-time and run-time integration	17	Where the time goes
6	Scope by priority	18	Build plan
7	System architecture, with the libraries and models used at each layer	19	Every formula the tool uses, with what each symbol means
8	The pipeline	20	The learning loop
9	Variant A's ten sections, as used in the worked example	21	Which paper shaped which part
10	The six steps	22	The eight targets
11	Share of each kind of content that Variant B keeps	23	Where each original requirement went
12	Told to answer in 50 tokens, a model wrote 157; given a sensible limit it wrote 86 [P10]	24	The results screen for the worked example

1 Executive summary

Most prompts are written the way people talk: a greeting, the request, the same rule said twice, a vague “put it in a table or something”, and a thank-you. A model reads and bills every word of that, and then writes a long, chatty answer that is billed at five times the rate. OptimusPrompt fixes both ends of that exchange without losing a single requirement.

What it does

You paste a prompt. OptimusPrompt reads it once, sorts each sentence into what it is — the task, a rule, the data, the requested format, or padding — and marks the words that must never change: numbers, names, allowed labels, “don't” and “never”, code. From that one sorted copy it writes two new prompts:

- **Variant A (high-context)** lays the prompt out in up to ten clear sections — who is answering, the goal, the data fenced off, the rules, the exact answer format, a self-check, and the key rules repeated at the end. It is for work where a wrong answer is expensive.
- **Variant B (low-token)** strips the prompt down in small, safe steps and caps how long the answer may be. It is for high-volume, routine calls.

Before either version is shown, four automatic checks confirm that every locked word is still there, every rule still holds, the goal has not drifted, and nothing new has been invented. If a check fails, the tool quietly steps back to the last version that passed. Alongside the two prompts you get exact token counts, the cost per call and per 10,000 calls, a table showing where every original requirement ended up, and a plain recommendation of which version to use.

Results on our worked example

The example is a deliberately wordy request to classify three food-delivery reviews (full text in Appendix A). Input tokens were counted with OpenAI's o200k tokenizer; answer lengths are estimates.

Measure	Original	Variant A	Variant B
Input tokens (measured)	184	245 (+33%)	**84 (−54%)**
Answer tokens (estimate)	~150	~110	~60
Total tokens	334	355 (+6%)	**144 (−57%)**
Cost per call	$0.001868	$0.001590	**$0.000768**
Cost per 10,000 calls	$18.68	$15.90	**$7.68**

Saving against the original	—	15%	**59%**
Answer writing time at ~42 tokens/s	~3.6 s	~2.6 s	**~1.4 s**
Token efficiency (times quality score Q)	2.99 Q	2.82 Q	**6.94 Q**
All four checks passed	—	yes	yes

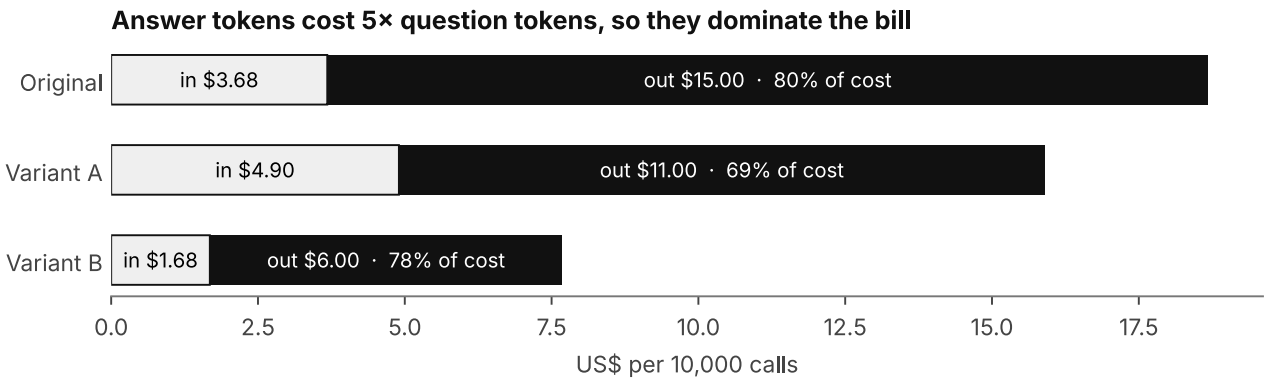


Figure 2. Where the money goes. On Claude Sonnet 5.5 an answer token costs $10 per million against $2 for a prompt token, so in the original prompt the answer is 45% of the tokens but 80% of the bill.

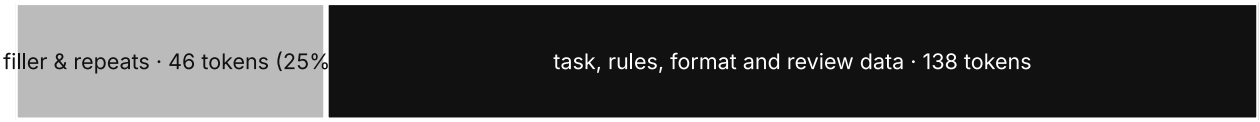
> **THE FINDING THAT SHAPED THE DESIGN**
>
> Variant A is a third *longer* than the original prompt and still 15% *cheaper*, because it tells the model exactly what the answer should look like and the chatty preamble disappears. Shortening the question helps; controlling the answer helps more.

We do not yet have a measured quality score for this example, so we report token efficiency as a multiple of quality rather than guess. Read that way, Variant B gives 2.3 times as much answer per token as the original at equal quality, and it stays ahead as long as its quality is at least 43% of the original's. Our target is 90%. At one million calls a month, this single prompt would drop from $1,868 to $768 — $1,100 saved before any batching or caching.

2 The problem and the people who have it

Waste in a prompt is easy to see and hard to remove safely. The obvious fixes — cut words, ask a model to “make it shorter” — save tokens on paper and quietly break the answer.

184-token original prompt — one quarter is padding



Five ways prompts go wrong

What happens	Why	How big it gets
Padded questions	Greetings, “could you please kindly”, rules said twice, vague format requests.	In our example, 46 of 184 tokens (25%) add nothing.
Padded answers	No stated format or length, so the model adds a preamble and a sign-off; easy tasks get long step-by-step reasoning they don't need.	Answers are 80% of our example's cost. Short drafted reasoning used about 40 tokens where full reasoning used about 200, at 91% vs 95% accuracy [P9]. Budgeting the answer cut output by 67% with under 3% accuracy loss [P10].
Slow, crowded context	Pasted documents and repeated examples fill the window and slow every call.	Removing half the context cut memory 36% and run time 32% with almost no quality change [P4]; other work reports 1.4–2.6× faster end-to-end [P6].
Rules lost in the middle	Models pay less attention to the middle of a long prompt.	Putting key information at the start and end beat middle placement by up to 21.4% on a question-answering test while using about a quarter of the tokens [P6].
Cutting too much	Blind trimming deletes labels, negations and numbers.	One study's harshest trimming more than doubled its efficiency score while matching answers fell by 44.5 points [P1]. A label “Arts & Culture” became “Culture” [P3]. Keyword lists did worse than no compression at all [P8].

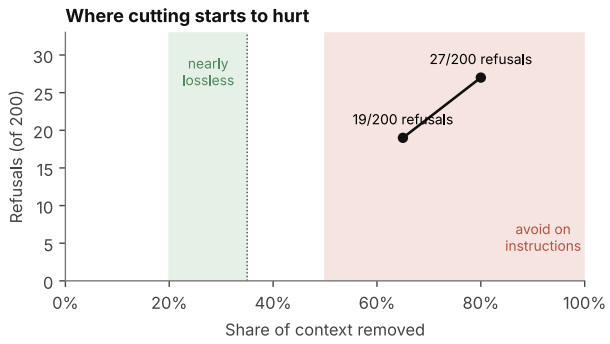


Figure 3. Removing 20–35% of context is almost free; past 50%, refusals climb [P4]. This sets our default cut.

What it costs

For a prompt template called N times a month, the waste is the tokens that could be removed without losing any requirement, multiplied by the price, multiplied by N. On our example that is $1,100 a month at one million calls. Because the price of an answer token is five times that of a question token, the answer side usually matters most.

Root causes and what we do about each

Cause	What OptimusPrompt does	Step
Conversational tone	Removes filler using a word list and a measure of how predictable each phrase is.	S4, S7
Saying a rule twice for emphasis	Finds near-identical sentences and keeps the most specific one.	S7
Instructions and data mixed together	Sorts every sentence by role and fences data off from rules.	S2, S6
No stated answer format	Writes an exact format and sets a sensible answer length with a floor.	S6, S7
Key rules buried mid-prompt	Puts the goal first and repeats the main rules at the end.	S6
“Just shorten it” with a model	Never rewrites freely. It extracts, rebuilds from the sorted copy, then checks.	S5, S9

Who it is for

Arjun — runs the LLM platform at a mid-size company

Persona 1 · Enterprise AI engineer / LLM operations lead

His situation	Owns 40+ prompt templates behind a shared gateway; 1–10 million calls a month; finance reviews the bill monthly.
What he needs	Lower cost per 10,000 calls with no drop in quality; stay inside his response-time limit; send easy requests down a cheaper path.
What gets in his way	Hand-edited prompts can't be reviewed properly; no proof that a shorter prompt kept every rule; token counts differ between providers; nothing catches a bad prompt before it ships.
He'll call it a success when	High-volume templates cost 40% less, quality holds at 90% or better, and the build logs show zero lost key words.

Meera — building her first AI feature at a startup

Persona 2 · Full-stack developer

Her situation	Prototyping on free credits; rewrites the same prompt dozens of times a day.
What she needs	A reliable, well-laid-out prompt fast; answers that keep the same shape; a cost estimate before launch.
What gets in her way	Trial and error; answers that drift in format or invent fields; a surprise bill after launch.
She'll call it a success when	Variant A works first time with an exact answer format, the warnings (“no output format given”) are fixed in one pass, and she can see the monthly cost at her expected volume.

Kavya — writes the business rules, doesn't write code

Persona 3 · Product manager / prompt designer

Her situation	Writes prompts for support triage and content tagging that carry company policy.
What she needs	Confidence that every policy rule survives; the trade-off between cost and quality in plain words.
What gets in her way	She can't tell if an engineer's “optimised” prompt still enforces a rule; explanations arrive in jargon.
She'll call it a success when	The coverage table shows every requirement ticked in both versions, and any assumption the tool added appears as an editable, highlighted placeholder.

3 How people will use it

Journey 1 — Improving one prompt by hand (Meera, Kavya)

#	What the person does	What the tool does	Typical time
1	Pastes a prompt; optionally adds context and examples, picks the target model, sets a quality-versus-cost slider.	Cleans the text, hides personal details, flags injected instructions (S1).	15 ms
2	—	Splits the prompt into labelled parts (S2) — see Figure 4.	≤ 0.8 s
3	—	Works out the task type, difficulty, risk and likely answer length (S3).	20 ms
4	Looks at a heat map of where the tokens went and a short list of warnings.	Counts tokens with the target model's own counter; marks the words to lock (S4, S5).	0.35 s, in parallel
5	—	Builds Variant A and Variant B at the same time (S6, S7).	≤ 1.2 s
6	—	Prices both and runs the four checks; steps back if any fail (S8, S9).	0.4 s
7	Compares three cards side by side, a red/green diff, the coverage table, and a recommendation.	Recommends A, B or the cascade, with tips such as an answer length cap (S10).	30 ms
8	Edits any highlighted assumption, then copies or exports.	—	—
	End to end		**≈ 2.4 s**

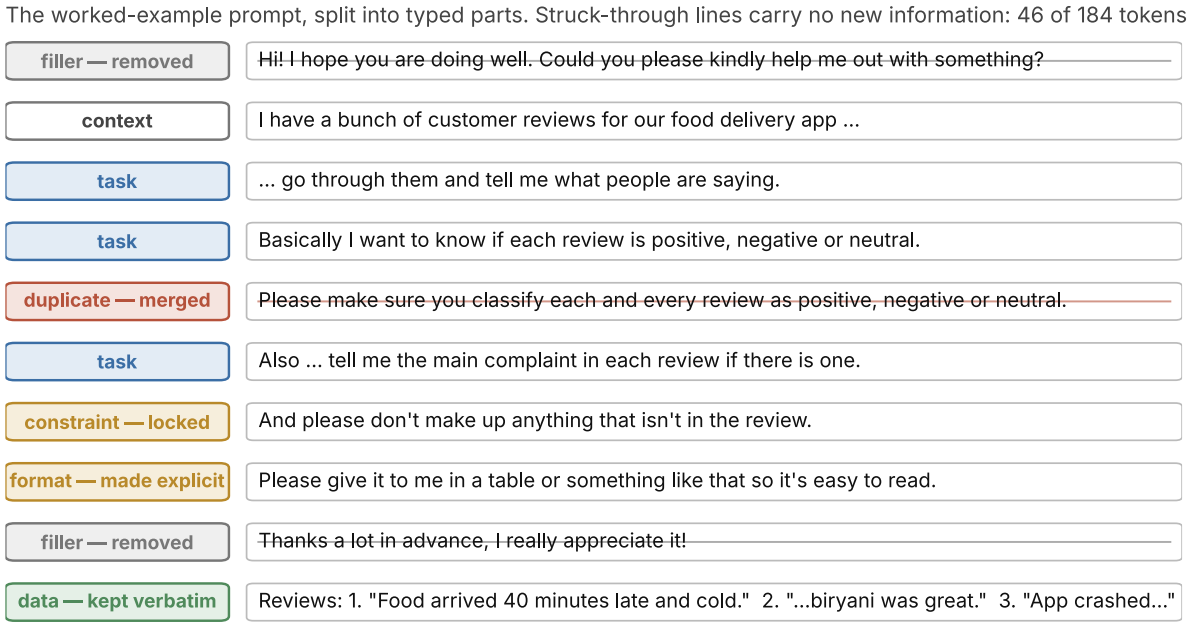


Figure 4. The worked example after step 2. Every sentence gets one label; padding and repeats are marked for removal, rules and data are locked.

Journey 2 — Checking prompts automatically in the build (Arjun)

Arjun's prompts live in a code repository. A command-line check sends all of them to the batch endpoint whenever someone opens a pull request. The merge is blocked if any version fails a check, if the prompt's quality score drops by more than 10 points, or if its token count grows by more than 10% against the main branch. Approved versions are stored under a fingerprint of the template, so the tool runs once per template change, not once per call. At run time the gateway fetches the stored versions and uses the cascade: send Variant B; if the answer fails a cheap check, resend with Variant A; log what happened.

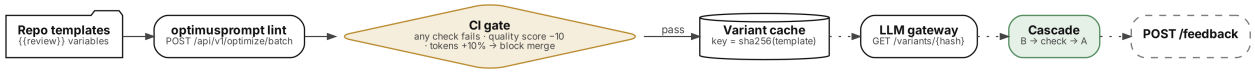


Figure 5. Build-time and run-time integration. Template placeholders such as `{{review}}` are treated as data and locked.

Journey 3 — Checking quality against real answers and learning from use

(Arjun)

1. **Verify mode.** The original, Variant A and Variant B are each run on up to five sample inputs. A separate judge model compares each new answer with the original's answer, twice with the order swapped so position doesn't sway it.
2. **Scoring.** Each version gets a win-or-tie rate, a pass/fail on answer format, and exact-match or test results where a correct answer is known. This fills in the quality score Q.

3. **Logging.** The task features, the version chosen, the judge score, the user's thumbs up or down, and any cascade escalation are stored.
4. **Learning to route.** Once 1,000 labelled cases exist, a learned router is trained. It replaces the hand-written rules only if, on held-back cases, it costs less without lowering quality.
5. **Better templates.** An offline search tries new wordings for Variant A and keeps one only if it scores better on the benchmark (Section 5, step S11).

4 What we will build, and in what order

Must-haves are everything needed to keep the promise “nothing important is lost” and to run the demo. Should-haves save more or explain better, but the promise holds without them. Nice-to-haves need real usage data or extra model calls, so they come last.

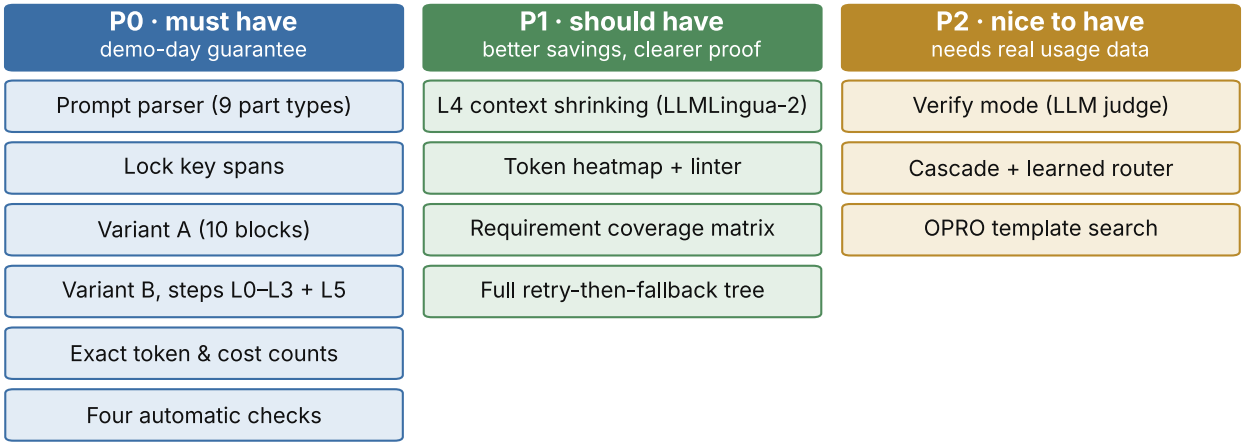


Figure 6. Scope by priority. The technical names of each component appear in Sections 5 and 6.

P0 Must have — hackathon scope

ID	Requirement	Done when
P0.1	**Prompt parser.** Splits the input into eight kinds of content — role, task, context, data, examples, rules, format, question — plus a ninth label for padding. Code, JSON and tables are kept whole.	Every character belongs to exactly one part and the parts rebuild the input exactly. No code block is ever split. Labelling scores at least 0.85 against 200 hand-labelled sentences. One model call per request.
P0.2	**Lock key words.** Numbers and units, names, allowed label sets, identifiers, code, quoted text, negations, the answer format and template placeholders.	Finds at least 98% of 500 planted examples. Every locked item appears unchanged in every version shown; a miss triggers a step back.
P0.3a	**Variant A.** Up to ten sections (Figure 9); a section appears only if the prompt supplies content for it or a gap needs flagging.	No more than 2.5 times the original length, or original plus 250 tokens for very short prompts. No new facts; every added assumption is visibly marked.
P0.3b	**Variant B.** Steps L0, L1, L2, L3 and L5 of the reduction ladder (Figure 10). Risky tasks — code, legal, medical, money, heavy numbers — stop at L2.	Each step is kept only if all checks pass, and the step reached is reported. Stays in full sentences, never a keyword list.

P0.4	**Token and cost counter.** Uses each provider's own counter; falls back to an approximate counter with a stated ±20% margin. Shows cost per call, per 10,000 calls and per month, plus a diff view.	Cost sums match hand calculation to six decimal places. Approximate counts are always labelled as such.
P0.5	**Four automatic checks** on every version (Figure 13), run locally at no API cost.	Run on 100% of requests; results returned with the response.

(P1) Should have — after the hackathon

ID	Requirement	Done when	Why not P0
P1.1	**Shrink long context (L4).** Keeps only the sentences that matter for the question and one representative example.	At most 10% new words in the shortened text; at least 2× shorter on context over 1,000 tokens; all checks pass.	Only helps long prompts.
P1.2	**Heat map and warnings.** Shows which phrases carry little information and flags common mistakes.	Draws in under 0.5 s for a 2,000-token prompt; at least eight warning rules.	Explains; doesn't change results.
P1.3	**Coverage table.** Every original requirement mapped to where it sits in A and B (Figure 23).	Every rule, format and task appears as a row; any “dropped” row blocks that version.	The checks already enforce this; the table shows it.
P1.4	**Full retry tree.** Retry once with the error fed back, then step back.	At most one retry on the fast path; the reason for any step back is shown.	P0 ships the simple step back.

(P2) Nice to have — once there is usage data

ID	Requirement	Done when	Why last
P2.1	**Verify mode.** Real runs judged by a second model.	Variant B wins or ties at least 90% of the time; finishes in 20 s.	Costs real API calls every time.
P2.2	**Cascade and learned router.**	Promoted only if it beats the rule-based router on held-back data.	Needs 1,000+ labelled cases.
P2.3	**Template search.** 2–3 rounds of four candidates, stop early.	Ships only if efficiency rises and Variant A quality does not fall.	Offline; can overfit [P14].

Out of scope for version 1: techniques that need access to model weights, fine-tuning the target model, guaranteeing the factual truth of the model's answer, image or audio prompts, and fetching prices live.

5 How it works

5.1 The parts and how they connect

A web app and a command-line tool both talk to one Python service. That service runs every request through eleven steps. Most of the checking uses small models that run on an ordinary CPU, so it costs nothing per call. Only two short calls go out to a large model, through a gateway that can reach any major provider or a model running offline on the laptop. Logs, stored prompt versions and feedback sit in a local database; prices and thresholds live in two settings files so they can change without touching code.

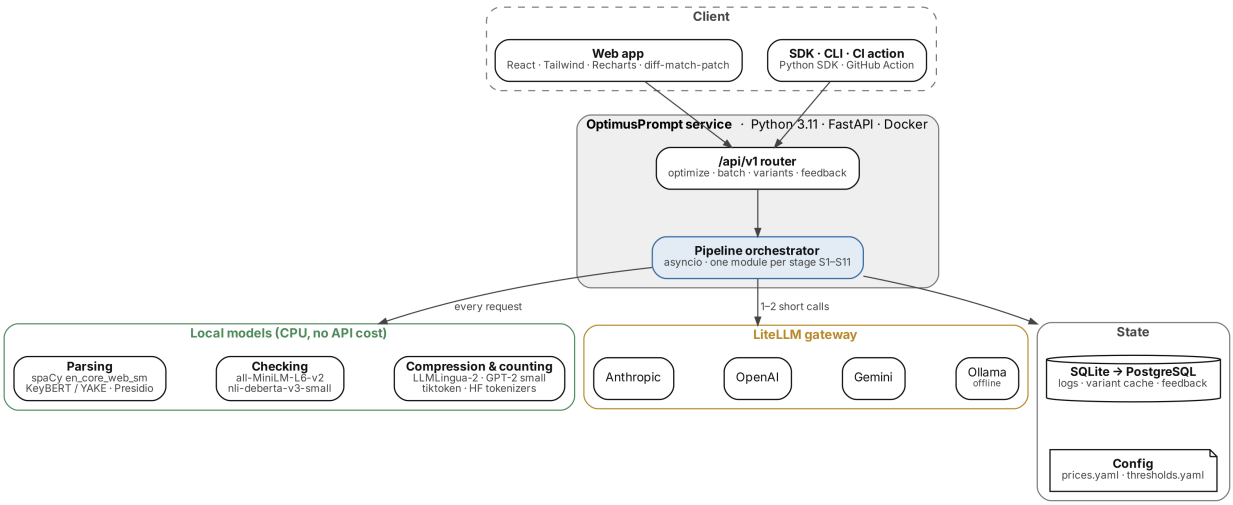


Figure 7. System architecture, with the libraries and models used at each layer.

5.2 The eleven steps

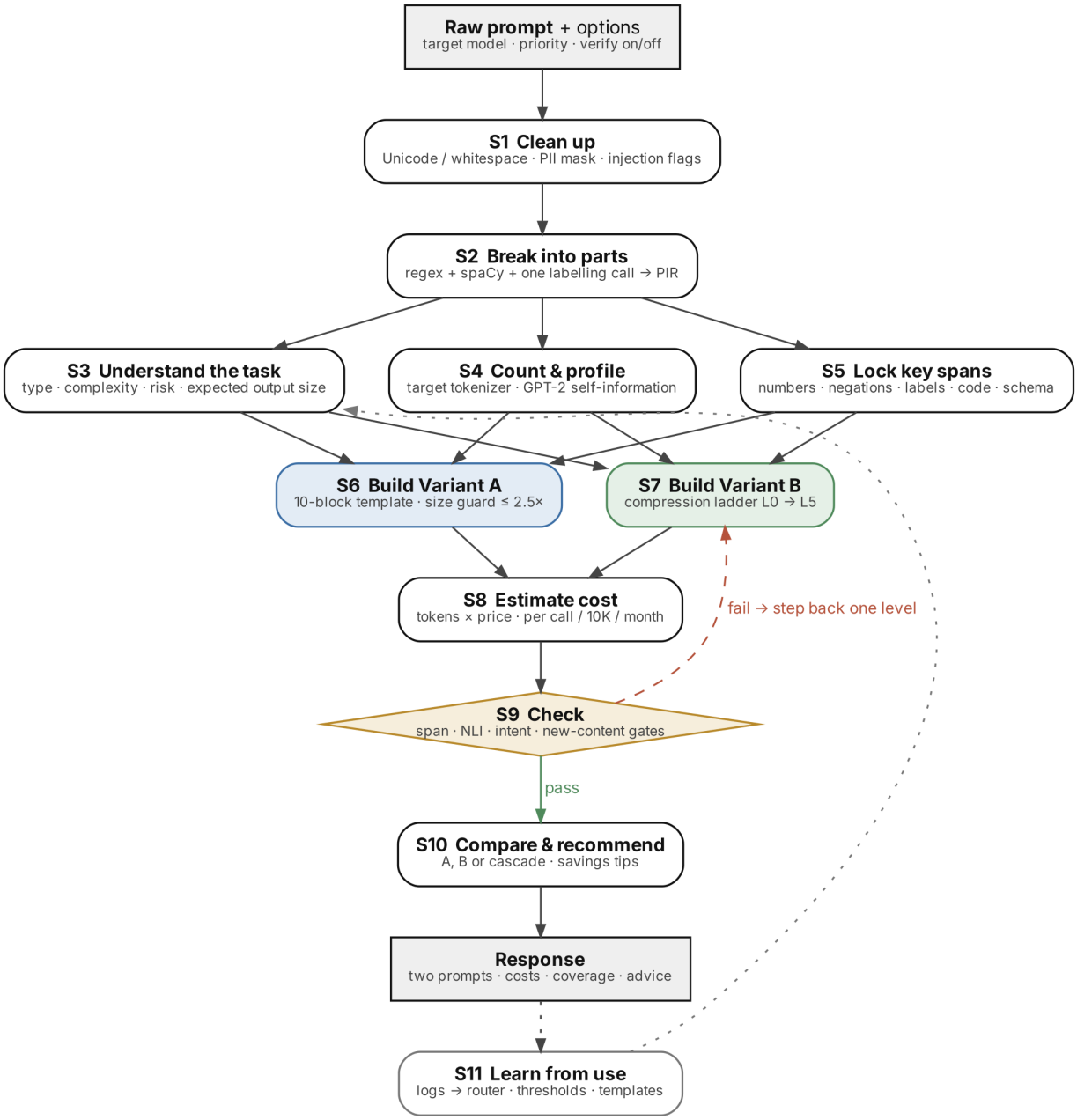


Figure 8. The pipeline. Steps 3–5 run side by side, as do 6 and 7. The dashed red line is the automatic step back; the dotted line is the learning loop.

S1 · Clean up

Normalises odd characters and spacing, detects the language, optionally swaps personal details for placeholders (and swaps them back in the final prompts), and spots text in pasted data that tries to give the model new orders. That text is fenced in as data and reported.

S2 · Break into parts

Code, JSON and tables are pulled out first and kept whole. The rest is split into sentences, and each sentence is labelled by simple rules plus a single model call. Sentences that say the same thing are linked together. Because both new prompts are built from this one labelled copy, any original requirement can be traced into either version.

S3 · Understand the task

What it decides	How
Task type	One of: code, debug, summarise, extract, classify, answer from documents, reasoning or maths, creative, translate, plan, chat.
Difficulty	A weighted count of sub-tasks, rules, reasoning words (“why”, “prove”, “step”) and length. Low, medium or high.
Risk	High if legal, medical or financial terms, code, or three or more numeric rules appear.
Gaps	Missing audience, format, length or scope. Each gap becomes a flagged assumption in Variant A.
Likely answer length	A table by task type adjusted by explicit cues (“in 50 words”, “list 5”, number of items). Replaced by a learned model once there is data.

S4 · Count and profile

Counts tokens with the target model's own counter. It also scores each phrase by how surprising it is to a small language model. Low-surprise phrases carry little information and are first in line for removal. That score drives the heat map. A quality score for the prompt itself (clarity, specificity, rules, structure, repetition) produces plain warnings such as “no answer format given”.

S5 · Lock key words

Combines pattern matching, name detection, a list of rule words (must, never, only, exactly, at most, not) and label-set detection. Locked words are handed to every later step as untouchable and checked again in S9.

S6 · Build Variant A

Five sections are copied straight from the labelled parts with no model involved: role, fenced inputs, examples, rules and the repeated tail. A model writes only the goal, assumptions, reasoning plan, answer format and self-check. It must keep every locked word, add no facts and change no requirement. Static instructions go first so that providers' prompt caching can apply on repeat calls.

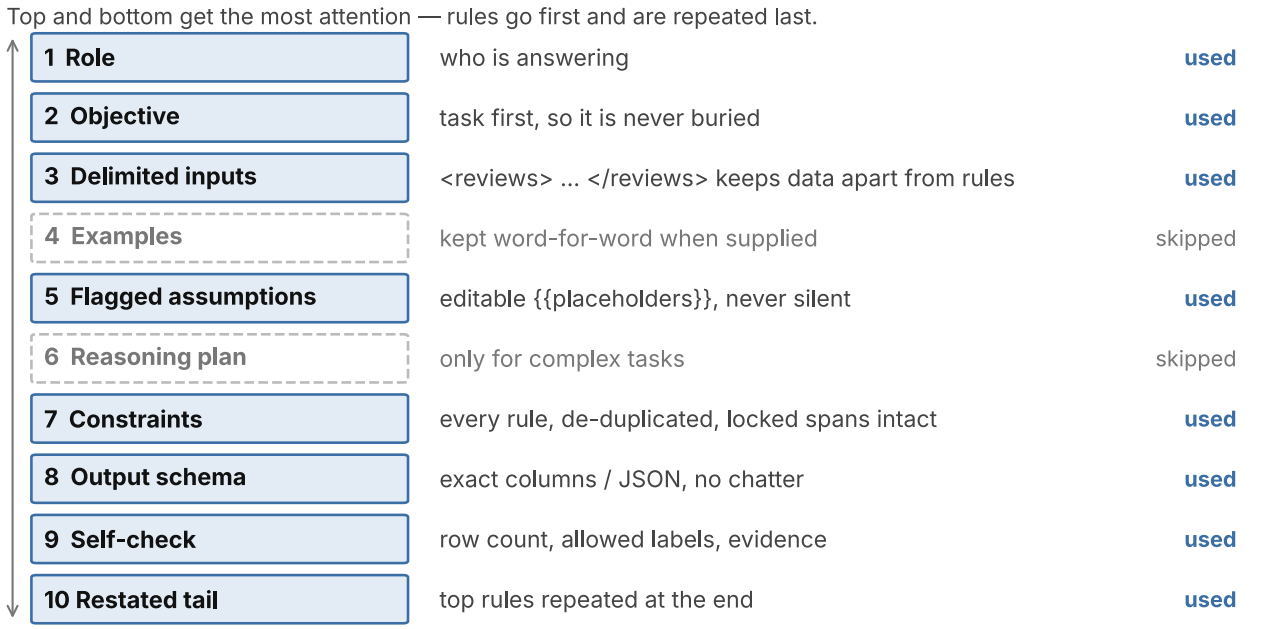


Figure 9. Variant A's ten sections, as used in the worked example. Two are skipped because the task is simple and no examples were given.

S7 · Build Variant B

Variant B climbs a ladder from the safest change to the riskiest. After each step the checks run; on the first failure the tool keeps the previous step's result.

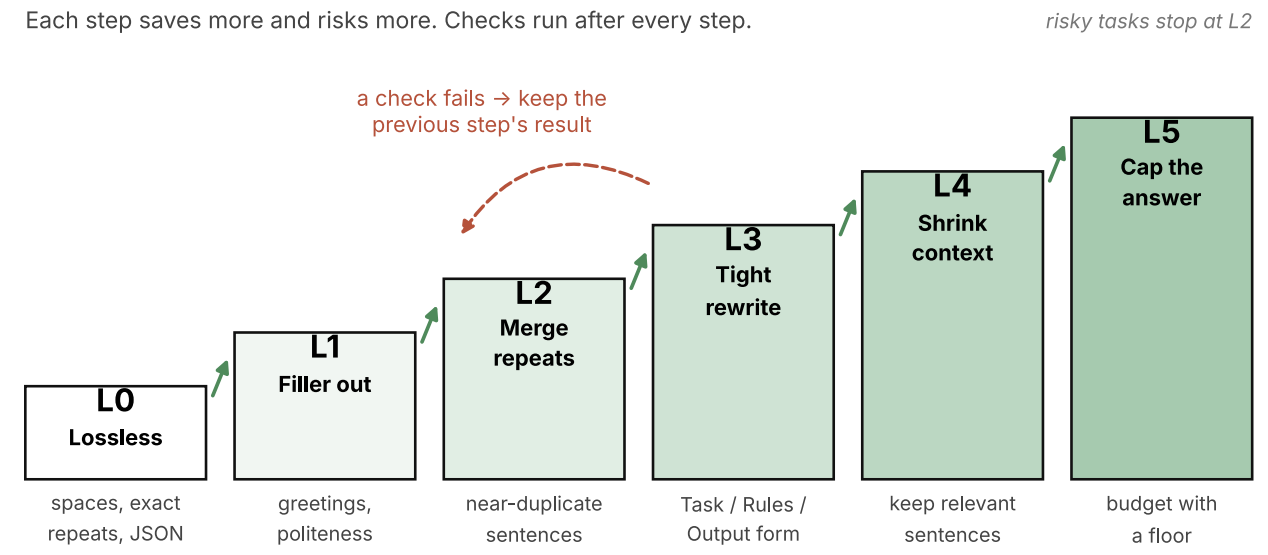


Figure 10. The six steps. L0–L2 are rule-based, L3 is one constrained rewrite, L4 uses an extractive compressor, L5 shapes the answer.

Different parts of a prompt can stand different amounts of trimming. Rules, format and data are never trimmed; the question and task only lightly; background and extra examples heavily. These starting shares follow published results [P5] and will be tuned from

usage. Ordinary prose is cut by 20–35% by default and never more than half for instructions [P4].

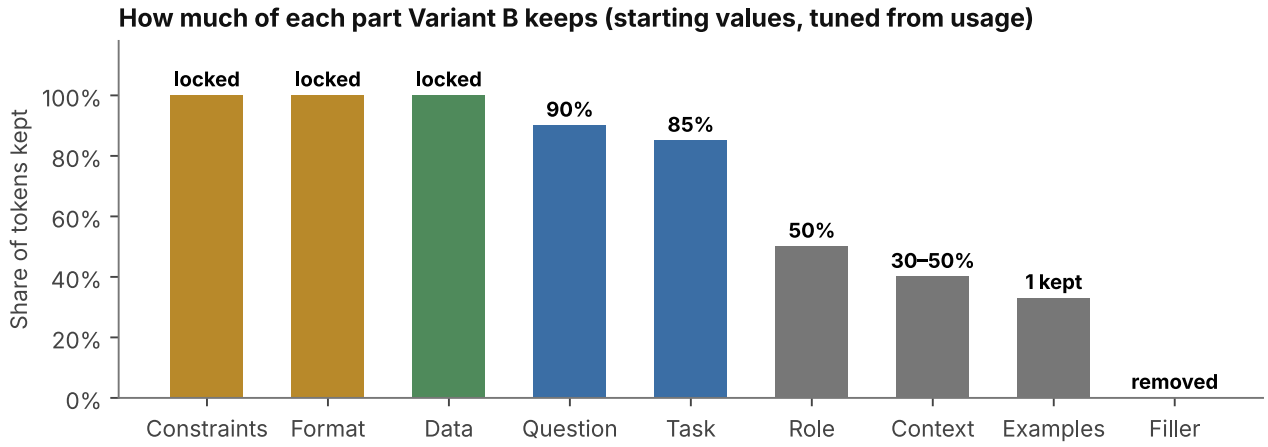


Figure 11. Share of each kind of content that Variant B keeps.

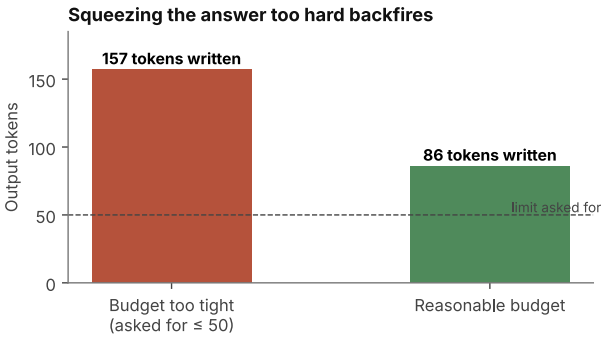


Figure 12. Told to answer in 50 tokens, a model wrote 157; given a sensible limit it wrote 86 [P10].

Capping the answer without backfiring. Step L5 asks for the answer only, in a compact format, with a length budget of 1.2 times the expected length — but never below a floor set per task type. The floor exists because an unreasonably tight limit makes the model ignore it and write *more*. The same figure, with 50% headroom, is suggested as the API's maximum-length setting.

For reasoning tasks we use short drafted reasoning, always with one worked example: without the example, accuracy fell from 90.4% to 65.5% in testing [P9].

S8 · Estimate cost

Cost is the input tokens times the input price plus the expected answer tokens times the output price, shown per call, per 10,000 calls and per month at the volume the user enters. Response time is estimated from the model's typical writing speed. The tool also counts what it spent itself, so the savings it reports are net, and it shows how many calls it takes to repay that. On the worked example, the optimiser's 1,210 tokens are repaid after about five calls (assuming a 900/310 split between input and output).

S9 · Check

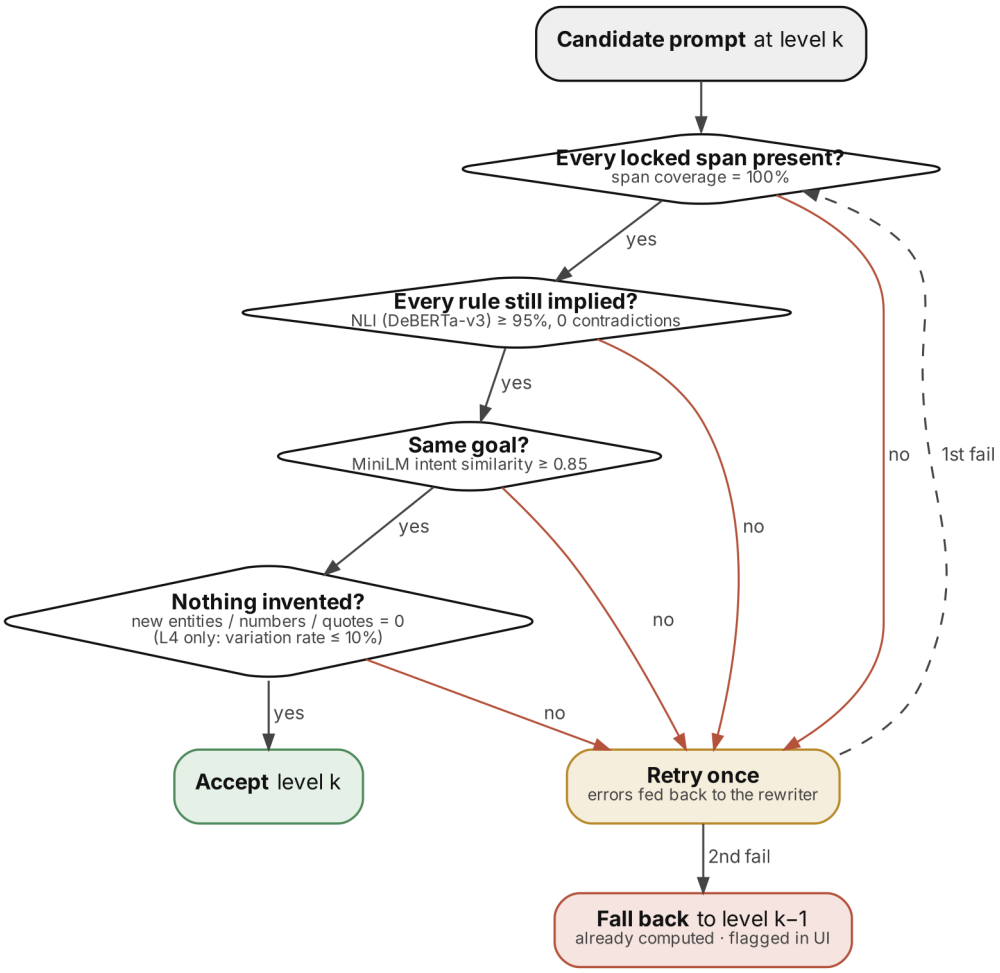


Figure 13. The four checks, in order. A failure gets one retry with the errors fed back, then a step back to the previous version, which is already built — so stepping back costs no extra time.

> **A CORRECTION WE FOUND WHILE TESTING**
>
> Our early notes applied the “at most 10% new words” test to every Variant B. A correct rewrite of the worked example scores 20% on that test, simply because it adds words like “output”, “Markdown” and “none”. The test was designed for extractive shortening, where any new word is a red flag [P7]. So we apply it only to step L4; rewrites are instead held to “no new names, numbers or quotes” plus the rule-coverage check.

S10 · Compare and recommend

Each version gets a score that rewards expected quality and penalises cost and risk, weighted by the user's slider. Simple rules sit on top: complex or high-risk tasks get Variant A; simple, high-volume tasks get Variant B; everything else gets the cascade. The trade-off text is generated from the numbers, never written freely. Tips cover the answer

length cap, batching (Claude's batch interface halves the price), caching the fixed part of the prompt, and a cheaper model tier.

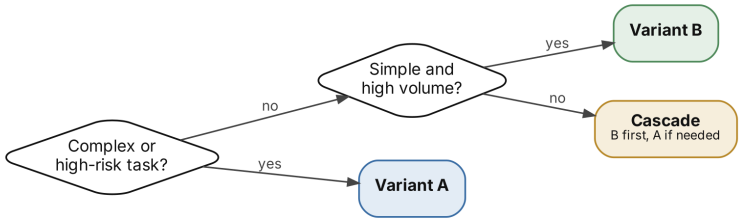


Figure 14. Recommendation rules.

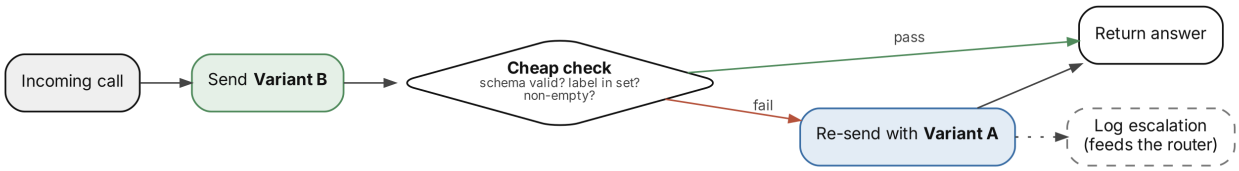


Figure 15. The cascade. On our example it beats always using A whenever B passes the cheap check on more than 48% of calls.

S11 · Learn from use

Every request's features, chosen version, judge score, user rating and any escalation are logged. These train the expected-length model, tune the check thresholds per task type, train the router, and drive the offline template search (Figure 20).

5.3 The interface

Five endpoints: optimise one prompt, optimise a batch of up to 100, fetch stored versions by fingerprint, send feedback, and list supported models with their prices. A failed check is not an error — the response comes back normally with a “stepped back” flag and the reason. If the large model is unreachable, the tool still returns rule-based versions and says so.

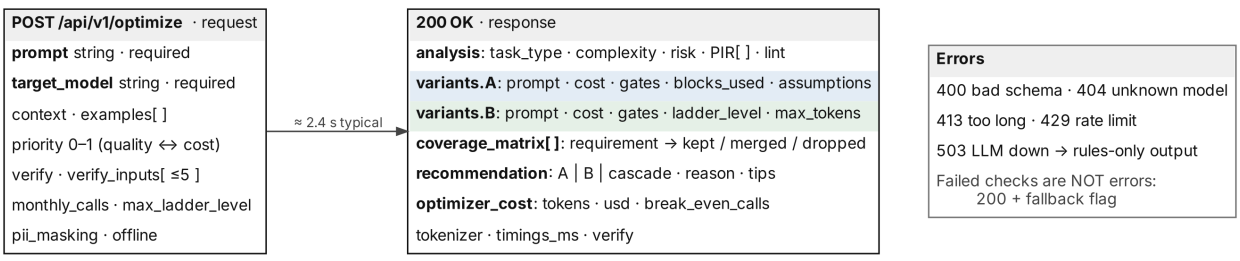


Figure 16. Request and response at a glance. The full machine-readable schema ships with the code.

5.4 Speed

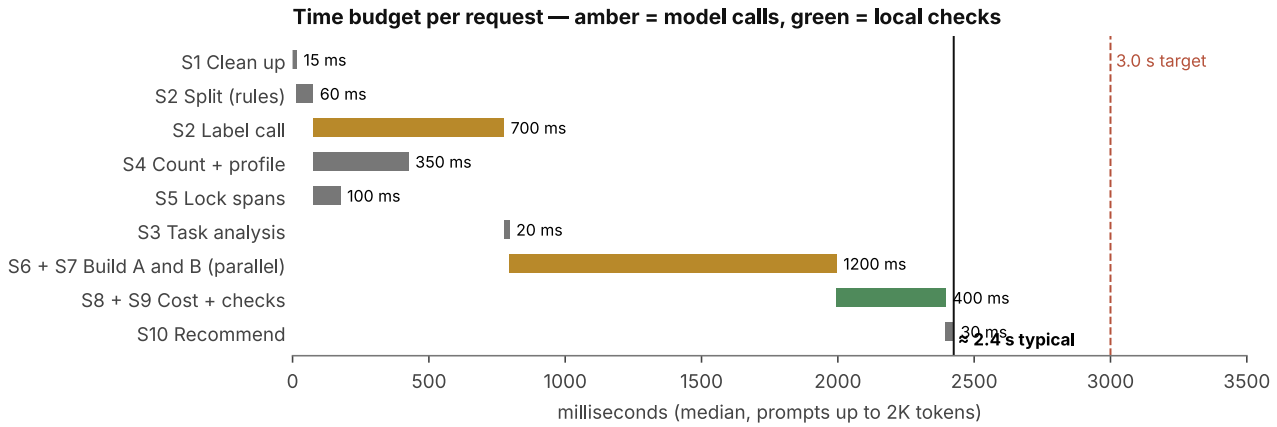


Figure 17. Where the time goes. Our target is a median of 3.0 s and a 95th percentile of 6.0 s for prompts up to 2,000 tokens.

> **WHAT THE SPEED TARGET DEPENDS ON**
>
> Claude Sonnet 5.5 writes about 42 tokens a second. At that pace the ~150 tokens the two builders produce would take 3.6 s on their own and miss the target. So the re-writing is done by the faster, cheaper tier (Claude Haiku 5.5), and we test its real speed in the first hour of the build. Building half of Variant A without a model is what keeps the model's share small.

5.5 Build plan

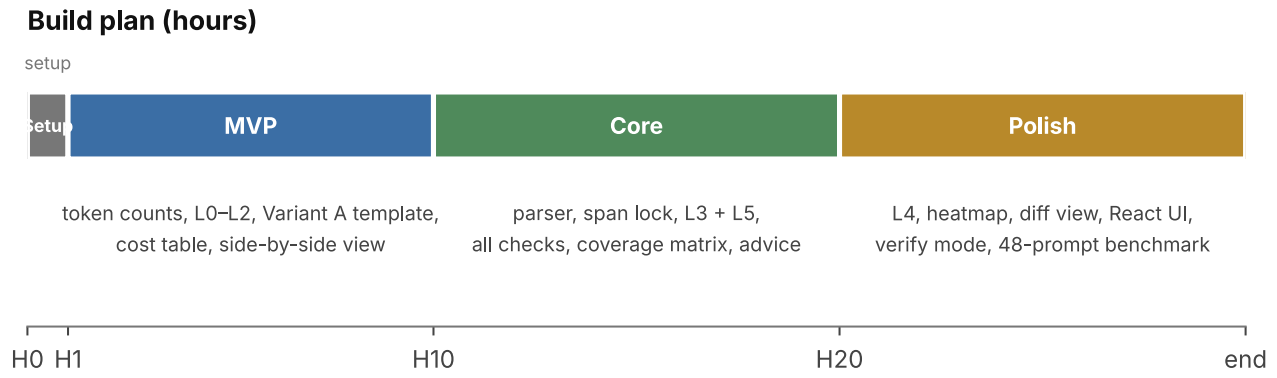


Figure 18. Build plan. Each phase ends with a test: after MVP the worked example must reproduce 184 / 245 / 84 tokens; after Core every P0 requirement must pass on 16 benchmark prompts; after Polish the targets in Section 7 are filled with measured numbers.

5.6 The formulas

Token efficiency

$$TE = \frac{Q}{T_{in} + T_{out}} \times 1000$$

Q = quality score (0–1) from the judge or benchmark

Cost of one call

$$C = T_{in} \cdot P_{in} + \hat{T}_{out} \cdot P_{out}$$

P = price per token from prices.yaml

Answer budget

$$B_{out} = \max(F_{task},\ \lceil 1.2\,\hat{T}_{out} \rceil)$$
$$max_tokens = \lceil 1.5\,B_{out} \rceil$$

F = floor per task type, so the budget never backfires

Rule coverage (NLI)

$$CC = \frac{1}{|C|} \sum_{c \in C} \mathbb{1}\left[\max_{s} P_{entail}(s \Rightarrow c) \geq 0.7\right] \geq 0.95$$

each rule c must follow from a sentence s of the new prompt

Span coverage

$$PSC = |P \cap v| \,/\, |P| = 1$$

every locked span P appears word-for-word in variant v

Recommendation score

$$S(v) = w_q\,\hat{Q}(v) - w_c\,\frac{C(v)}{C(raw)} - w_r\,R(v)$$

weights come from the quality ↔ cost slider and task risk

When the cascade pays

$$\pi_{pass} > \frac{C(B)}{C(A)} = \frac{0.000768}{0.001590} = 0.48$$

B must pass the cheap check on > 48% of calls

Break-even of the tool itself

$$N_{be} = \left\lceil \frac{C_{opt}}{C(raw) - C(B)} \right\rceil$$

calls needed before the optimiser's own cost is repaid

Figure 19. Every formula the tool uses, with what each symbol means. Starting values for the thresholds are in Appendix B.

6 What makes it different

Can it…	Ask a model to “shorten it”	A compressor on its own	A prompt linter	OptimusPrompt
Keep a structured copy of the prompt	no	no	partly	**yes**
Prove numbers, labels and “don't” survived	no	partly	no	**yes**
Give two versions for two kinds of job	no	no	no	**yes**
Shorten answers without backfiring	no	no	no	**yes**
Put key rules where models notice them	no	partly	no	**yes**
Step back automatically when a cut goes too far	no	no	no	**yes**
Show where every requirement went	no	no	no	**yes**
Report its own cost, so savings are net	no	no	no	**yes**

1. It rebuilds instead of retelling

Asking a model to shorten a prompt means it writes a new one and hopes nothing was dropped; published work shows it drops reasoning steps [P5]. OptimusPrompt takes the prompt apart, rebuilds both versions from the same labelled pieces, then checks them against those pieces. Every requirement has an ID, so a loss is visible instead of silent.

2. It can prove nothing important was lost

The two worst failures in the research are replacing names with codes (−30 points) and boiling a schema down too far (−44.5 points) [P1], plus labels getting truncated [P3]. Locking those words and re-checking them on every request turns “we were careful” into a yes-or-no test.

3. It shortens answers, not just questions

Most compression tools only touch the prompt, yet answer tokens cost five times as much. OptimusPrompt budgets the answer, with a floor because over-tight limits backfire [P10], and uses short drafted reasoning only alongside a worked example [P9].

4. It puts rules where models look

Variant A states the goal first and repeats the key rules last, the arrangement that beat burying them in the middle [P6]. Putting fixed instructions first also lets provider caching do its job.

5. It recovers on its own

One study found that adapting how hard to cut, rather than using one fixed strategy, gave the best balance: 8.7% fewer input and 5.5% fewer output tokens with 88.4% matching answers [P1]. We apply the same idea to each step of the ladder: go as far as the checks allow, and no further.

6. It routes between its own versions

The idea of trying the cheap option first and escalating on failure comes from model selection [P11]; we apply it to prompt versions, with a clear point at which it pays off.

7. It gets better with use

Usage trains the router [P12], tunes the thresholds and drives the template search [P14]. More use means better routing and more savings at the same quality — something that can't be copied from the code alone.

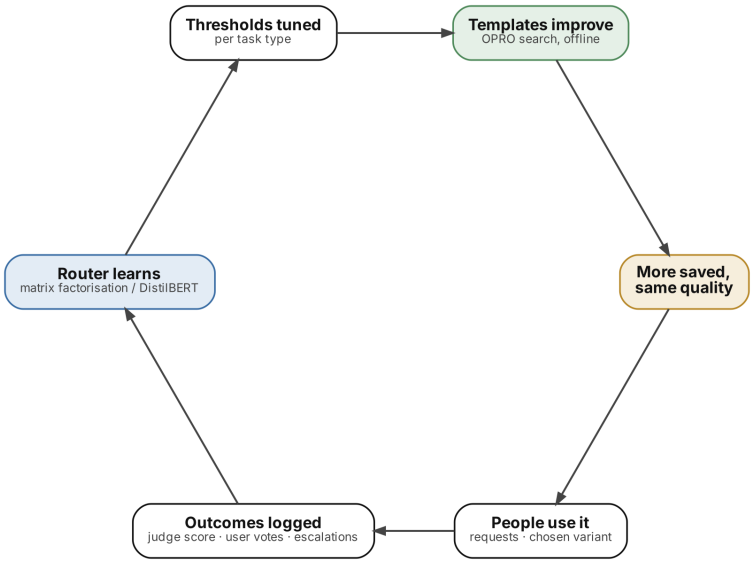


Figure 20. The learning loop.

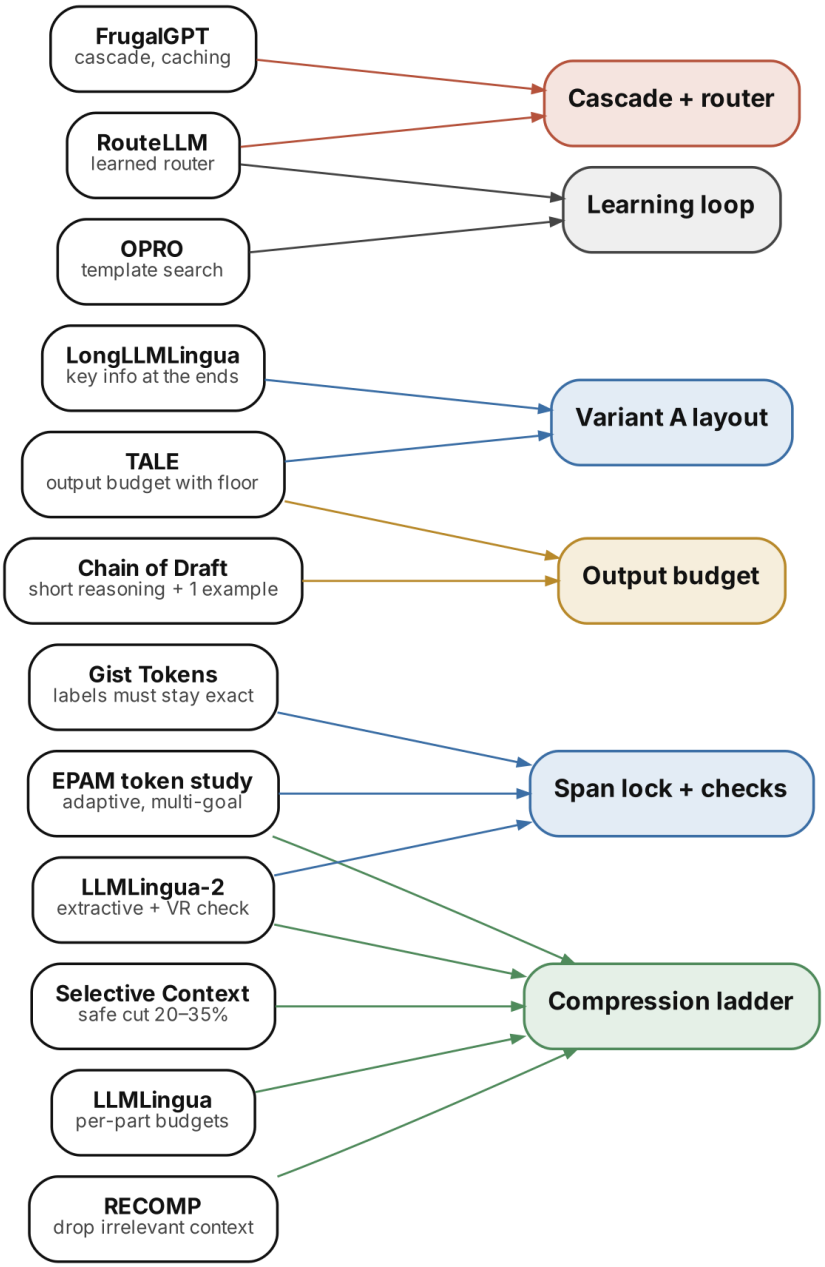


Figure 21. Which paper shaped which part. Each box gives the one finding we took from it.

7 How we will measure success

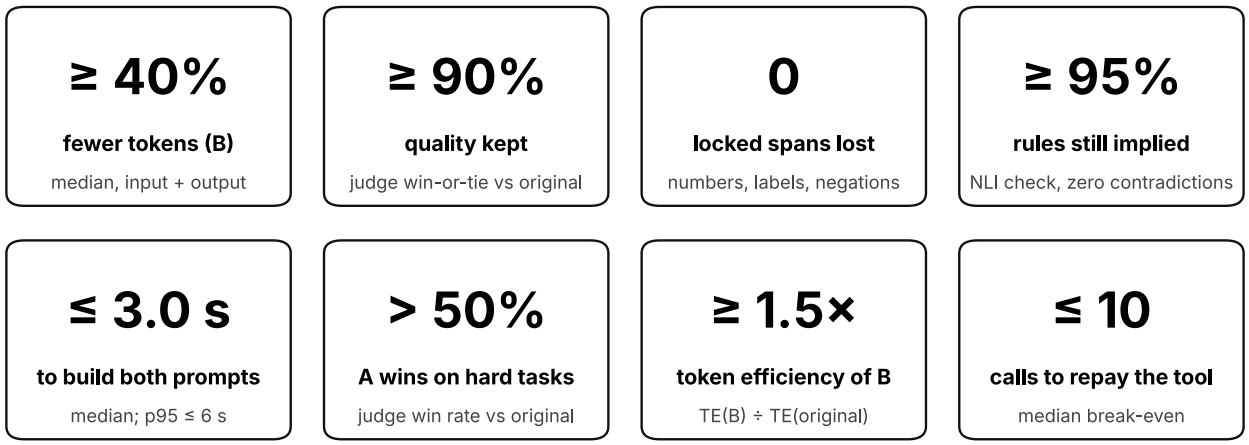


Figure 22. The eight targets. The claim “at least 40% cheaper at 90% quality with nothing lost” is published only if the first three hold on both test models.

What each number means

Measure	Definition	Where the target comes from
Token reduction	One minus Variant B's total tokens (prompt + answer) divided by the original's. Median across the benchmark.	Published results of 50–67% [P4, P10]; 57% on our example.
Quality kept	Share of comparisons where the judge rates the new answer as good as or better than the original's, checked in both orders.	Light trimming matched the original in [P1].
Locked words lost	Count of locked items missing from any version shown to a user.	Our own guarantee.
Rules kept	Share of original rules that some sentence of the new prompt implies, with no contradictions.	The S9 check.
Speed	Time to analyse and build both versions, not counting verify mode.	Figure 17 time budget.
Variant A on hard tasks	Judge win rate against the original on high-difficulty prompts.	[P6, P14]
Token efficiency	Variant B's efficiency divided by the original's (formula in Figure 19).	Follows from the first two: 0.9 ÷ 0.6 = 1.5.
Break-even	Calls before the tool's own cost is repaid.	About 5 on our example.

The benchmark

Prompts. 48 in total: six in each of eight areas — code, extraction, summarisation, classification, answering from documents, reasoning and maths, creative writing, and planning. Each area has two easy, two medium and two hard prompts, and at least two with long context so that step L4 is exercised. Where a correct answer exists (classification, extraction, maths, code with tests) we record it.

Runs. Original, A and B on all 48 prompts on two models — one strong, one cheap — at temperature 0: 288 answers. The judge compares A and B against the original in both orders: 384 judgements. A win counts only if both orders agree; otherwise it is a tie.

Statistics. Medians with 95% confidence intervals from 10,000 bootstrap resamples; a paired signed-rank test for token and cost savings; results reported separately per model and per area.

What we switch off to test each idea

Switched off	What it tells us
Ladder capped at each step in turn	How much each step saves and what it costs in quality.
Word locking	Whether locking is what prevents lost words (we expect losses without it).
The answer-length floor	Whether the floor prevents the backfire in Figure 12.
Short drafted reasoning	Whether it saves answer tokens without breaking the quality target.
Repeating rules at the end	Whether placement improves how well Variant A's rules are followed.

8 Assumptions, limits and risks

What we are assuming

- **We only change the words.** The target model is used through its normal prompt interface. We do not need its weights, so we can't use methods that compress prompts into vectors [P2].
- **“Quality” means “as good as the original's answer”** when no correct answer is known, and a judge model stands in for a human rater. We improve the prompt; we don't promise the model's answer is true.
- **Answer lengths are estimates** until verify mode or the benchmark measures them, and are labelled as such.
- **Prices come from a settings file**, dated, not fetched live. Prices that change with prompt length (such as Claude Haiku 5.5) are stored for each band.
- **Assumptions the tool adds are always visible** and editable, never slipped in quietly.

How tokens are counted when the official counter isn't available

If the provider offers a counter, we use it and the count is exact. If we're offline but have that model's counter cached, we use the cached one. Otherwise we fall back to a general-purpose counter and label the result as approximate, with a ±20% margin. Our worked example was counted this way. Claude's own counter usually gives a higher number for the same text, so absolute counts will move, but we expect the percentage savings to hold — and we will check that.

Limits

- Prompts from about 20 to 8,000 tokens; longer ones must be split by the caller in version 1.
- English first. The context shrinker works in many languages but is largely untested on Indian languages, and our name detector is English-only.
- The rule-coverage check uses a small local model. It may sometimes miss a rule that was reworded and wrongly reject a good version. That costs savings, not safety, because the tool steps back.
- The 3.0 s target depends on the faster model tier writing at least 100 tokens a second.

Risks

Risk	Likely?	Impact	What we do
Rule check too strict	Medium	Smaller savings	Tune its threshold per task type from logs; failures fall back safely.

Judge model biased by order or length	Medium	Misleading quality score	Judge both orders; call it a tie when they disagree; use exact answers or tests where they exist.
Rewriter ignores the locked words	Low–medium	Lost words	The span and new-content checks are rule-based, so such a version is never shown.
Instructions hidden in pasted data reach the rewriter	Medium	Corrupted version	Flag and fence them in step S1; tell the rewriter that fenced text is data, not orders.
Prices go out of date	High over time	Wrong cost shown	Prices live in a file with a “last checked” date shown on screen.
Template search over-fits the benchmark	Medium	Worse templates	Stop early; hold back a set of prompts; ship only if Variant A quality doesn't drop.

A The worked example in full

Original prompt — 184 tokens

```
Hi! I hope you are doing well. Could you please kindly help me out with something? I have a bunch
of customer reviews for our food delivery app and I want you to go through them and tell me what
people are saying. Basically I want to know if each review is positive, negative or neutral.
Please make sure you classify each and every review as positive, negative or neutral. Also it
would be really great if you could tell me the main complaint in each review if there is one. And
please don't make up anything that isn't in the review. Please give it to me in a table or
something like that so it's easy to read. Thanks a lot in advance, I really appreciate it!

Reviews:
1. "Food arrived 40 minutes late and cold."
2. "Delivery partner was polite, biryani was great."
3. "App crashed twice while paying, but the food was okay."
```

Variant A — 245 tokens, 8 of 10 sections

```
You are a customer-experience analyst for a food delivery app.

Objective: Classify the sentiment of each customer review and identify its main complaint, so the
product team can prioritise fixes.

<reviews>
1. "Food arrived 40 minutes late and cold."
2. "Delivery partner was polite, biryani was great."
3. "App crashed twice while paying, but the food was okay."
</reviews>

Rules:
- Sentiment must be exactly one of: positive, negative, neutral.
- A review with both praise and a complaint is "neutral" unless one side clearly dominates.
[Assumption, edit if needed]
- Main complaint: a short phrase taken from the review text, or "none".
- Use only what the review says. Do not infer or invent details.

Output format: a Markdown table with columns | # | Sentiment | Main complaint | Evidence (quote)
|, one row per review, in the original order. No text before or after the table.

Before answering, check: every review has exactly one row, every label is one of the three
allowed values, and every complaint appears in its review.

Reminder: three labels only; complaints only from the review text.
```

Variant B — 84 tokens, steps L0 → L3 plus L5

```
Classify each review: positive/negative/neutral, plus main complaint from its text (or "none").
Don't invent details. Output only a Markdown table: | # | Sentiment | Main complaint |

Reviews:
1. "Food arrived 40 minutes late and cold."
2. "Delivery partner was polite, biryani was great."
3. "App crashed twice while paying, but the food was okay."
```

ID	Requirement from the original	Variant A	Variant B
R1	Classify each review	✓ kept	✓ kept
R2	Only 3 labels allowed	✓ kept + self-check	✓ kept
R3	Main complaint, if any	✓ kept, "none" explicit	✓ kept, "none" explicit
R4	Don't make anything up	✓ kept + repeated at end	✓ kept
R5	Easy-to-read table	✓ exact column schema	✓ exact column schema
R6	All 3 reviews word-for-word	✓ kept, in <reviews> tags	✓ kept
R7	Mixed reviews (gap found)	✓ added, flagged	– not added

Figure 23. Where each original requirement went. R7 is a gap the tool found; Variant A adds it as a flagged assumption, Variant B leaves it out to stay short.

Checks. Both versions: every locked word present, no new names, numbers or quotes, all rules still implied. Variant B's new-word share is 20%, which is expected for a rewrite and is not tested at this step.

Recommendation. Variant B for bulk, repeated classification of short reviews; Variant A for long or mixed reviews and audits that need quoted evidence. Tips: cap answers at 90 tokens, send about 20 reviews per call, and use the batch interface for anything not interactive.

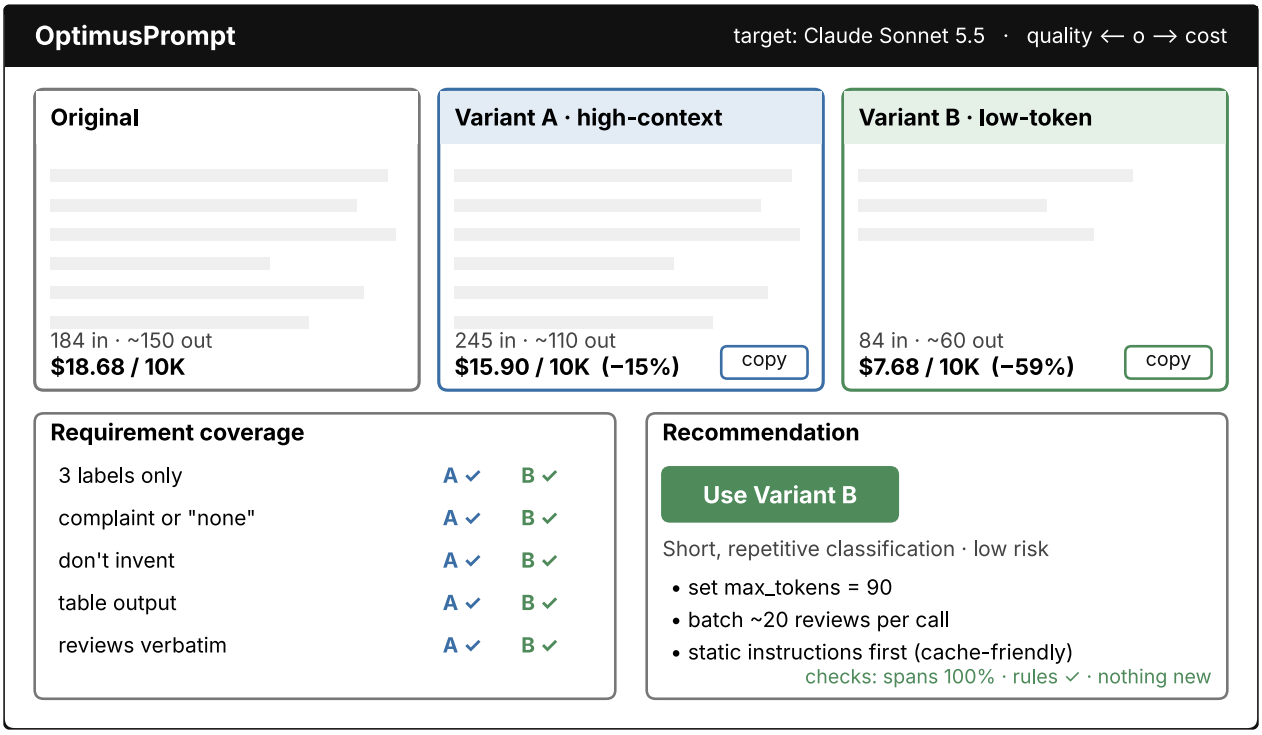


Figure 24. The results screen for the worked example.

B Starting settings

Prices (October 2026)

Model	Input, $ per million	Output, $ per million	Note
Claude Sonnet 5.5	2.00	10.00	Anthropic list price; batch interface 1.00 / 5.00
Claude Haiku 5.5	0.10 up to 100K tokens, 0.50 above	0.50 up to 100K, 2.50 above	Priced by prompt length; recheck before demo
Claude Opus 5.5	4.00	20.00	From team notes; recheck before demo
Other providers	per file	per file	Added the same way

Minimum answer length by task

Task	Floor (tokens)
Classify	larger of 40 or 15 per item
Extract	60
Answer from documents	50
Summarise	80
Reasoning / maths	100
Code	150
Plan / analyse	150
Creative	200

Check thresholds

Setting	Start value
Locked words present	100%
Rules still implied	≥ 95%
Per-rule confidence	≥ 0.70
Contradiction cut-off	< 0.50
Same-goal similarity	≥ 0.85
New words, step L4 only	≤ 10%
“Same sentence” similarity	> 0.90
Variant A size limit	2.5× or +250
Answer budget / cap	1.2× / 1.5×
Retries on the fast path	1
Cases before learned router	1,000

C References

[P1]	O. Grynets, D. Babarytskyi. *Token Optimization Strategies for LLM-Based Oracle-to-PostgreSQL Migration.* EPAM Systems.
[P2]	Z. Li, Y. Liu, Y. Su, N. Collier. *Prompt Compression for Large Language Models: A Survey.* NAACL 2025. arXiv:2410.12388.
[P3]	J. Mu, X. L. Li, N. Goodman. *Learning to Compress Prompts with Gist Tokens.* NeurIPS 2023. arXiv: 2304.08467.
[P4]	Y. Li, B. Dong, C. Lin, F. Guerin. *Compressing Context to Enhance Inference Efficiency of Large Language Models* (Selective Context). EMNLP 2023. arXiv:2310.06201.
[P5]	H. Jiang et al. *LLMLingua: Compressing Prompts for Accelerated Inference of Large Language Models.* EMNLP 2023. arXiv:2310.05736.
[P6]	H. Jiang et al. *LongLLMLingua: Accelerating and Enhancing LLMs in Long Context Scenarios via Prompt Compression.* ACL 2024. arXiv:2310.06839.
[P7]	Z. Pan et al. *LLMLingua-2: Data Distillation for Efficient and Faithful Task-Agnostic Prompt Compression.* Findings of ACL 2024. arXiv:2403.12968.
[P8]	F. Xu, W. Shi, E. Choi. *RECOMP: Improving Retrieval-Augmented LMs with Compression and Selective Augmentation.* ICLR 2024. arXiv:2310.04408.
[P9]	S. Xu, W. Xie, L. Zhao, P. He. *Chain of Draft: Thinking Faster by Writing Less.* 2025. arXiv:2502.18600.
[P10]	T. Han et al. *Token-Budget-Aware LLM Reasoning* (TALE). Findings of ACL 2025. arXiv:2412.18547.
[P11]	L. Chen, M. Zaharia, J. Zou. *FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance.* 2023. arXiv:2305.05176.
[P12]	I. Ong et al. *RouteLLM: Learning to Route LLMs with Preference Data.* ICLR 2025. arXiv:2406.18665.
[P14]	C. Yang et al. *Large Language Models as Optimizers* (OPRO). ICLR 2024. arXiv:2309.03409.

Pricing: Anthropic Claude API pricing as listed in October 2026 (Sonnet 5.5, Haiku 5.5). Sonnet 5.5 writing speed of about 42 tokens per second from published third-party measurements. Papers 3 and 13 in the original pack were the same file, so 13 distinct sources are cited.